

6 Conclusion & Outlook

This work contributes to the study of the GRL problem. We have provided a reduction to handle large state and action spaces by sequentializing the decision-making process. This helped us improve the upper bound on the number of states in ESA from an exponential dependency in $|\mathcal{A}|$ to logarithmic. The gain is *double exponential* in terms of the action-space dependence at no other cost.

Our result carries a broader impact on the implementation of *general* RL agents¹². The required storage for such agents, which have access to a non-MDP, approximate Q-uniform abstraction, can be reasonably bounded which only scales logarithmically in the size of the action-space.

We conclude the paper with some future research directions. This work analyses the case when the agent has a fixed aggregation map. [?] provides an outline for a learning algorithm to learn such abstractions which can be combined with our sequentialization framework.

Another direction, which we also did not touch in this work, is to explore the connection, if any, between the surrogate-MDPs of a map on the original environment, and its extension on the sequentialized problem. By lifting the small binary ESA map, say ψ , back to $\mathcal{H}$, one obtains a small map directly on $\mathcal{H}$, say ϕ . While ψ used sequentialization/binarization for the construction of ϕ , the map ϕ can be used without further referencing to sequentialization. This suggests that a bound logarithmic in $|\mathcal{A}|$ should be possible without a detour through the sequentialization. This deserves further investigation.

We sequentialize the action-space through an *arbitrary* coding scheme C , so the main result does not depend on this choice. Sometimes, it is possible that the action-space may allow natural sequentialization, e.g. in a video game controller the macro action might be a binary vector where the first bit might represent the left/right direction, the second bit indicates up/down, and so on. The exact nature of these binary decisions depends on the domain which is reflected by the choice of encoding C . Sequentialization was our path to double-exponentially improve that bound. Whether there are more direct/natural aggregations with the same bound is an open problem. Moreover, if the agent is learning an abstraction through interaction, the choice of these functions may become critical.

This paper focused on rigorously formalizing and proving the main improvement result. One can also try to empirically show the effectiveness of our improved upper bound. To do this, we need a problem domain where ESA requires more states than the sequentialized/binarized version of it. But a point of caution is that the upper bound still scales badly in terms of γ and ε . Any reasonable value of these parameters would imply a huge upper bound. Even with Markovian abstractions, a cubic dependency on the discount factor is the best achievable. We considered a general underlying process and non-Markovian abstractions, and dramatically improved the previously best bound $(1 - \gamma)^{-3|\mathcal{A}|}$ to $(1 - \gamma)^{-3 \cdot 2}$. Indeed it would be interesting to see whether this can be further improved to the optimal $(1 - \gamma)^{-3}$ rate.

Acknowledgements. This work has been supported by Australian Research Council grant DP150104590. Thanks to the anonymous reviewers for their feedback and to Andrs Gyrgy who pointed out that the sequentialized/binarized process in Figure 2 preserves the Markov property, which encouraged us to also consider the Markov case.

Esse fugiat asperiores veniam tenetur similique praesentium facere, mollitia repellat dolorem maiores atque tenetur porro, ab ea voluptas eius corrupti atque nesciunt ex, animi atque eum officiis suscipit voluptatum molestiae, mollitia in praesentium doloribus modi hic molestias debitis inventore

¹²The general (or strong) agents are designed to work with a wide range of environments [?].

sint perferendis. Iure voluptas minus eligendi quibusdam incidunt porro quia iste corporis voluptates, in recusandae veniam autem odio perspiciatis consequuntur officia voluptatem, earum recusandae harum eius officia?